Upgrading Kubernetes clusters with kubeadm

Tim Warner

Principal Author | IT Ops | Pluralsight

TechTrainerTim.com

Four framing questions

What does kubeadm upgrade plan tell you before you even touch the infrastructure?

What is the correct sequence for upgrading the control plane?

How does the drain, upgrade, uncordon pattern protect running workloads?

How do I verify the cluster is healthy at every stage of the upgrade?

Checking in with Globomantics

“Security patched a critical CVE in the kubelet last week. We need every node upgraded to v1.35 by Friday. Our etcd backup is our safety net, but I need zero downtime -- our SLA doesn't allow a maintenance window.”

Marisol C, Platform Engineer, Globomantics Robotics Company

What does kubeadm upgrade plan tell you before you touch anything?

Pre-upgrade reconnaissance with kubeadm upgrade plan

Shows current version, available target versions, and pre-flight results

Validates that kubeadm binary matches the target version

Reports missing packages or configuration issues before you commit

Prints the exact kubeadm upgrade apply command to run next

kubeadm upgrade plan is read-only -- changes nothing on the cluster

What is the correct sequence for upgrading the control plane?

Control plane upgrade: the six-step process

Step 1	apt-mark unhold kubeadm, then apt install kubeadm=target-version
Step 2	kubeadm upgrade apply v1.35.x on the control plane node
Step 3	apt-mark unhold kubelet kubectl, then apt install both at target version
Step 4	systemctl daemon-reload, then systemctl restart kubelet
Step 5	apt-mark hold kubeadm kubelet kubectl
Step 6	kubectl get nodes to verify control plane shows new version

How does the drain, upgrade, uncordon pattern protect running workloads?

Worker node upgrade: drain, upgrade, uncordon

kubectl drain node --ignore-daemonsets --delete-emptydir-data evicts pods gracefully

Pods are rescheduled to other Ready nodes before the drain completes

Upgrade kubelet and kubectl on the drained node (same apt sequence)

systemctl daemon-reload, then systemctl restart kubelet

kubectl uncordon node re-enables scheduling on the upgraded node

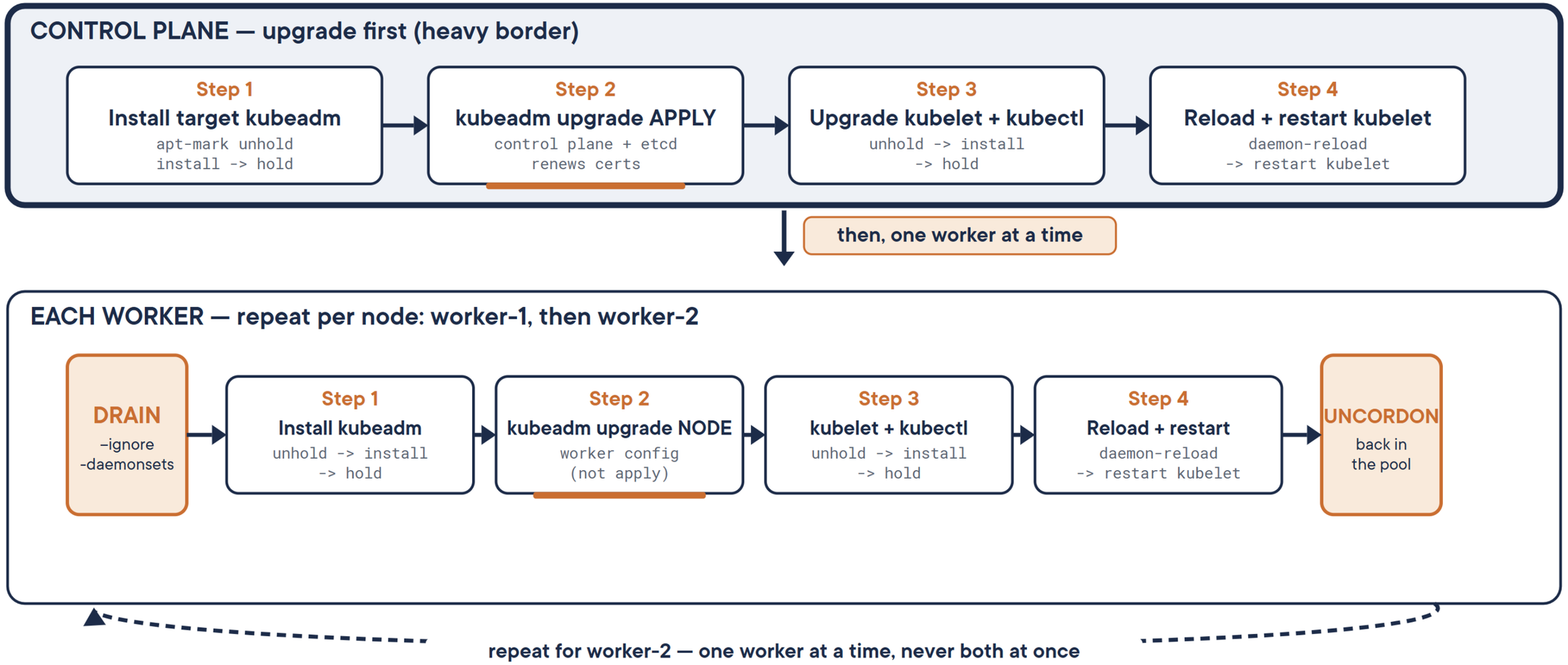
How do I verify the cluster is healthy at every stage of the upgrade?

Verification checklist at every upgrade stage

- kubectl get nodes: all nodes show new version and Ready status
- kubectl get pods -A: all pods Running, no CrashLoopBackOff or Pending
- kubectl version: server version matches target
- kubectl -n kube-system get pods: control plane components healthy
- Deploy a test workload to confirm the cluster accepts new pods

The kubeadm upgrade sequence: one picture

Read top to bottom: control plane first, then ONE worker at a time.



Control plane runs kubeadm upgrade APPLY (once). Every worker runs kubeadm upgrade NODE.

Heavy border = control plane. Amber tab = the drain / uncordon wrapper workers add. The labels carry the meaning, not the color.

Demo

Upgrade a three-node cluster from v1.34 to v1.35 with zero downtime

Checking out with Globomantics

“All three nodes at v1.35, all pods Running, and we did not drop a single request. The CVE is patched and the SLA held. That etcd backup never needed to be used -- which is exactly the outcome we wanted.”

Marisol C, Platform Engineer, Globomantics Robotics Company

From Globomantics to You

Always run kubeadm upgrade plan first

It's read-only reconnaissance that catches problems before you commit to anything

Control plane upgrades before workers

Kubernetes supports one-version skew (control plane ahead of workers) but never the reverse

Drain, upgrade, uncordon is the worker pattern

This three-step sequence protects running workloads and is a standard CKA exam task

Verify at every stage, not just at the end

kubectl get nodes after each node upgrade -- do not batch verification